LPG prediction

Name:Prathiksha Harish Baleri

USN:4CB22CB044

Domain: Data science

---

[31]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import datetime as dt
```

[32]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
```

[33]:
```
df = pd.read_excel("india_lpg_crisis_dataset_2026.xlsx")
df.head()
```

[33]:

	Date	City	State	LPG_Availability	Commercial_Cylinder_Price_INR	Domestic_Waiting_Time_Days	Import_Status	Supply_Status
0	2026-03-13	Kolkata	West Bengal	High	3918	22	Reduced Imports	Normal
1	2026-03-02	Chennai	Tamil Nadu	Medium	2690	14	Reduced Imports	Crisis
2	2026-03-06	Lucknow	Uttar Pradesh	Critical	3868	22	Normal	Alert
3	2026-03-11	Ahmedabad	Gujarat	Medium	3039	24	Reduced Imports	Normal
4	2026-02-04	Chennai	Tamil Nadu	High	3622	25	Normal	Crisis

[34]:
```
df = df[['Date','Domestic_Waiting_Time_Days','Commercial_Cylinder_Price_INR']]
df.columns = ['date','waiting_time','price']
```

[35]:
```
df['date'] = pd.to_datetime(df['date']).map(dt.datetime.toordinal)
df.head()
```

[35]:

	date	waiting_time	price
0	739688	22	3918
1	739677	14	2690
2	739681	22	3868
3	739686	24	3039
4	739651	25	3622

Here we uploaded the lpg dataset

Converted to ordinal data

predict price

```
[36]: X = df[['date','waiting_time']]
      y = df['price']
```

```
[37]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

```
[38]: lr = LinearRegression()
      lr.fit(X_train, y_train)
```

```
[38]: ▾ LinearRegression
      LinearRegression()
```

```
[39]: y_pred = lr.predict(X_test)
```

```
[40]: y_pred
```

```
[40]: array([3136.62134183, 3116.53179746, 3181.16724791, 3210.05246689,
             3128.90961656, 3037.58786578, 3146.83126226, 3156.67986623,
             3121.25993124, 2967.83194513, 3128.54830011, 3052.19562351,
             3130.74721877, 3243.27348323, 3059.90734878, 3166.22919371,
             3009.09498322, 3101.89301976, 3156.31854978, 3050.75035772,
             3175.71648124, 3170.2346946 , 3219.53975442, 3095.2962638 ,
             3085.80897627, 3145.35497649, 3122.01358411, 3101.89301976,
             3229.3883584 , 3128.94063654, 3138.82026049, 3070.87092207,
             3047.10617328, 3056.98579724, 3096.08093664, 3023.37244447,
             3072.64648433, 3101.17038687, 2962.35015849, 3139.90420983,
             3102.64667263, 3159.9937542 , 3014.57676986, 3180.44461502,
             3071.23223852, 3162.5539893 , 3014.57676986, 3229.02704195,
             3077.76695452, 3236.3464308 , 3193.21477054, 3020.45089293,
             3118.73071612, 3208.21486469, 3145.35497649, 3041.23205022,
             3121.62124768, 3086.89292561, 3139.90420983, 3210.02144692])
```

```
[41]: mean_squared_error= mean_squared_error(y_test, y_pred)
```

```
[42]: mean_squared_error
```

```
[42]: 271123.13728575315
```

Here we predict the price of the lpg cylinder in upcoming days

Model 2

model 2

predicting wait time

[43]:
```
X = df[['date','price']]
y = df['waiting_time']
```

[44]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

[45]:
```
lr1 = LinearRegression()
lr1.fit(X_train, y_train)
```

[45]:
```
▾ LinearRegression
LinearRegression()
```

[46]:
```
y_pred = lr1.predict(X_test)
y_pred
```

[46]:
```
array([17.4072792 , 17.36552518, 16.31677645, 17.63610876, 17.68884197,
       16.6133016 , 16.08219508, 17.12832845, 16.21523475, 17.41701751,
       16.94831405, 17.28626086, 17.0007216 , 17.47771707, 16.88368978,
       16.01888016, 16.57893107, 17.68269937, 16.00881952, 17.83030058,
       17.04096417, 17.06841383, 17.48869627, 17.64943277, 16.52809676,
       16.44635067, 16.63715221, 16.29717957, 16.86912155, 16.76718571,
       17.62493751, 17.30220357, 17.3490614 , 17.6457786 , 17.76999182,
       16.70046713, 16.80345179, 16.63296695, 16.03443542, 16.17394336,
       17.05384227, 17.33213834, 17.58573038, 16.52841908, 16.70903304,
       17.38055603, 16.66537679, 17.67655342, 16.03528215, 17.66303067,
       16.32971634, 16.84304303, 16.4853006 , 16.637214  , 17.21831143,
       17.55593591, 16.37147035, 16.26986352, 16.09990309, 17.4737339 ])
```